Technical & Enterprise: CybAI Threat Detection Engine

Author: MJeat
Publication Date: 17th August 2026
Documentation Type: Public
GitHub: https://github.com/MJeat/LLM-For-Software-Security-Public

==========

Introduction

Project Nickname: CybAI
VM OS: REMnux
Model Type: LightGBM, Logistic Regression
System Type: Basic Web UI Version
Scan Competence: Files and URLs
Main Features: 6 file types scanner, PE files, pre-defined reasoning

---

1. Problem Statement

Current enterprise security pipelines suffer from three major operational bottlenecks:

- Zero-Day Vulnerability: Static hash lookups and YARA rules provide sub-millisecond execution but are entirely blind to novel, obfuscated, or zero-day threats.
- Resource & Financial Inefficiency: Routing every incoming file and URL through heavy cloud LLMs consumes prohibitive compute infrastructure, generates excessive API costs, introduces latency, and creates data privacy compliance risks.
- Tool Fragmentation & Analyst Fatigue: Operating isolated security tools forces SOC analysts to manually correlate outputs across disparate systems, delaying incident response times.

---

2. Proposed Solution: The CybAI Architecture

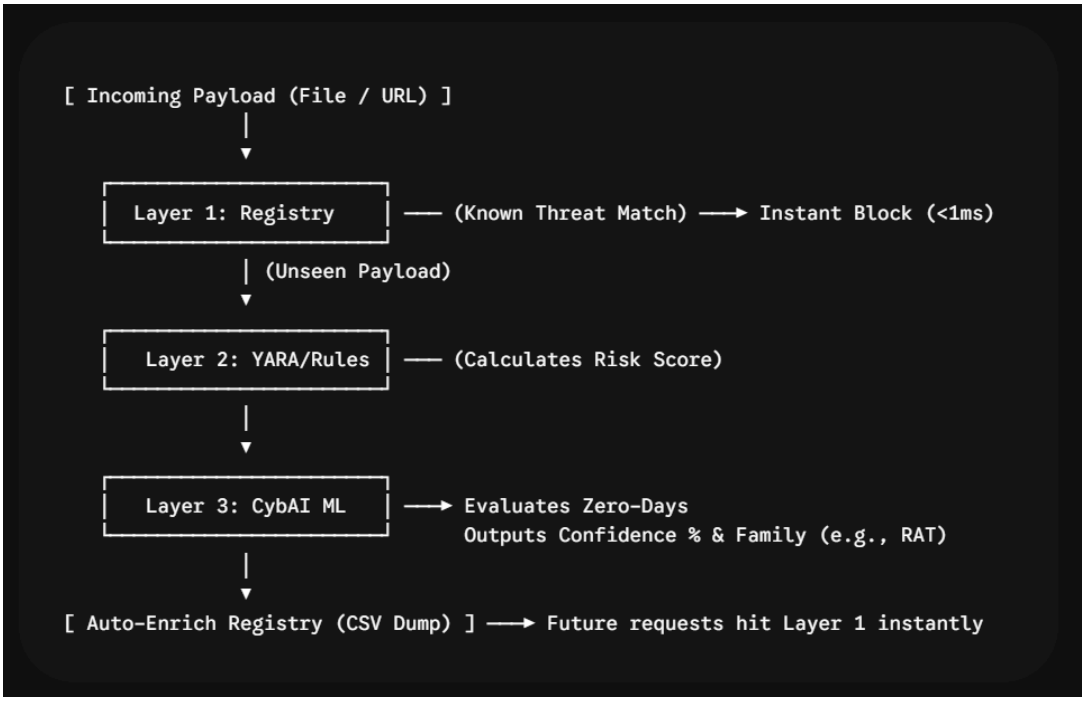
CybAI unifies file and URL scanning into an on-premise, 3-Layer Defense-in-Depth pipeline. Rather than executing heavy machine learning algorithms on every payload, CybAI uses a cascading execution model to minimize hardware consumption while guaranteeing zero-day coverage.

Security Layer	Inspection Mechanism	Operational Role	Target Performance
Layer 1: Threat Registry	Local Database & Blacklists	Instantly drops known malicious hashes and phishing URLs without consuming ML compute.	< 1 ms response
Layer 2: Heuristic Engine	Custom YARA Rules & Pattern Matching	Evaluates structural metrics and extracts risk scores. Does not issue final verdicts alone.	Heuristic scoring baseline
Layer 3: CybAI ML Engine	Hybrid ML Models (EMBER / Char-Ngram Vectorizers)	Evaluates complex structural features to classify zero-day threats, calculate confidence %, and attribute malware families.	High-confidence classification

---

3. System Architecture & Resource Optimization Model

The fundamental constraint of enterprise Machine Learning deployment is hardware overhead. CybAI addresses this through a cascading execution flow:



By automatically writing Layer 3 zero-day detections back into the local Layer 1 registry, the system self-learns in real time, ensuring that an unknown file is only processed by the ML model once.

---

4. Proof-of-Concept (PoC) Demonstration Protocol

To validate the architecture during the live demonstration, the testing workflow will execute as follows:

1. Unseen Threat Ingestion: Download a novel malware sample using automated fetch scripts to simulate an incoming zero-day attack.
2. Initial System Scan: Pass the sample through scanner.py. Demonstrate that it bypasses Layer 1 (unknown hash), receives a baseline risk score from Layer 2, and is flagged by Layer 3 with a high-confidence verdict and family classification (e.g., AsyncRAT).
3. Automated Registry Enrichment: Show that the scan engine automatically logs the newly identified threat hash to malware_registry_dump.csv.gz.
4. Instant Re-Scan Verification: Run the scan on the exact same sample a second time to demonstrate that Layer 1 catches the threat in < 1 ms, bypassing Layer 3 and preserving system resources.

---

5. Risk Assessment & Strategic Trade-Offs

Strengths:

- On-Premise Privacy: Zero external network dependency; proprietary corporate data never leaves the local environment.
- Attribution Depth: Goes beyond binary classification by providing specific family names (e.g., Ransomware, InfoStealer).
- Hardware Efficiency: Cascading design drastically reduces GPU/CPU utilization compared to full-scale cloud LLM evaluation.

Limitations & Mitigations:

- **Model Drift**: ML classifiers can lose efficacy over time as attacker tactics evolve.
    - **Mitigation**: Scheduled periodic retraining using logged threat telemetry.
- **Obfuscated / Packed Binaries**: High entropy packing can obscure structural features.
    - **Mitigation**: Layer 2 pre-processors handle basic binary unpacking prior to Layer 3 vectorization.

=====X=====